SPIDER-MAN 2D — GUÍA BÁSICA DEL PROYECTO

Explicación sencilla del Workflow, los Archivos y el Sistema de Sprites para defender el trabajo

1. ¿Cómo está dividido el proyecto? (Las 3 Carpetas)

Para que el proyecto sea ordenado y fácil de entender, está separado en **3 carpetas principales** (igual que una aplicación profesional):

1. Carpeta frontend/

Es lo que ve y juega el usuario. Contiene la página web, el diseño estilo cómic, el código del juego en Phaser 3 y todas las imágenes (sprites).

2. Carpeta backend/

Es el puente o mensajero (PHP). Recibe los pedidos del juego (como pedir la tabla o guardar un puntaje) y habla con la base de datos.

3. Carpeta database/

Es donde se guardan los datos (SQL). Tiene el código que crea la base de datos `spiderman_db` y la tabla `puntajes` en MariaDB.



2. El Workflow de la Página (¿Qué pasa paso a paso?)

El **workflow** (flujo de trabajo) es el camino que sigue la página desde que la abrís hasta que terminás de jugar y anotás tu récord:

1. **Abrís la página en el navegador:** Se carga `frontend/index.html` junto con `style.css`. El juego arranca en **pausa** detrás de una ventana que muestra los controles y la **Tabla de Posiciones**.
2. **Se cargan los puntajes de la Base de Datos:** Apenas carga la página, `main.js` usa `fetch()` para pedirle al archivo `backend/api/puntajes/index.php` la lista de los mejores jugadores guardados en MariaDB. El backend los devuelve ordenados primero por **más puntos** y, si hay empate, por **menor tiempo**.
3. **Cargan las imágenes (Preload):** Mientras ves la tabla, Phaser lee el listado de 440 imágenes de `assets_data.js` y las deja listas en la memoria para que el juego no se trabe al jugar.
4. **Empezás a jugar:** Al tocar "*¡Comenzar Partida!*", se oculta el cartel inicial, arranca el **temporizador desde 00:00.0** y empieza el bucle del juego (que actualiza la pantalla 60 veces por segundo).
5. **Exploración, Combate y Jefes:** A medida que avanzás, `city.js` va armando nuevos edificios y 1 botiquín por cada bloque (que reaparece cada 45 segundos). Matás enemigos comunes para sumar puntos. Cuando llegás al puntaje de un Jefe (ej. 1.000 pts para Kingpin), **dejan de salir enemigos comunes** y el mapa se cierra a **2 pantallas** con barreras rojas hasta que derrotes al jefe.
6. **Fin de la partida y Registro:** Si te quedás sin vida, si mantenés apretada la `R` durante 3 segundos, o si vencés al último jefe (Doctor Doom a los 40.000 pts), el reloj se detiene y aparece el formulario para escribir tu **Nombre y Apellido**. Al guardarlo, el backend lo anota en MariaDB y te lleva automáticamente a la pantalla de inicio con la tabla actualizada.

3. ¿Qué hace cada archivo del proyecto? (Guía Rápida)

A. Carpeta frontend/ (Lo visual y la lógica del juego)

Archivo	¿Qué hace explicado de forma simple?
`index.html`	Es el esqueleto de la página. Tiene el recuadro donde vive el juego (`#game`), la ventana de inicio con la tabla, y la ventana final con el formulario de Nombre y Apellido.
`style.css`	Es la decoración visual. Le da el estilo de historieta de Marvel, colores oscuros, bordes rojos brillantes, medallas de oro/plata/bronce en la tabla y diseño de los botones.
`src/config.js`	Guarda los números fijos del juego: el tamaño de la pantalla (1280x720), la altura del piso de la calle (`STREET: 1400`), la gravedad, la vida de Spider-Man (100) y los 45s del botiquín.
`src/assets_data.js`	Es el catálogo o diccionario del juego. Tiene la lista de los **10 enemigos comunes**, los **9 jefes** (con sus vidas y puntajes de aparición) y las rutas de las **440 imágenes PNG**.
`src/textures.js`	Dibuja por código el cielo nocturno, la luna, las ventanas iluminadas de los edificios y el dibujo pixel-art del botiquín médico blanco con la cruz roja.
`src/city.js`	Construye la ciudad infinita en bloques de 1.000 píxeles (llamados *chunks*). Pone el piso, los edificios donde se pega la telaraña y controla que el botiquín cure el 100% y reaparezca a los 45s.
`src/player.js`	Controla todo lo que hace **Spider-Man**: moverse, trepar paredes, colgarse con **clic derecho** (física de péndulo), golpear con **clic izquierdo**, tirar telaraña con **F** y esquivar con **Shift**.
`src/enemies.js`	Es el "cerebro" (Inteligencia Artificial) de los enemigos y jefes. Les dice cómo perseguirte, cuándo disparar, cómo volar (Duende Verde, Hobgoblin, Doom) y evita que se hundan en el piso.
`src/ui.js`	Dibuja los textos e indicadores arriba de la pantalla mientras jugás: tu barra de vida, los puntos, el cronómetro en vivo, cuánto falta para el próximo jefe y la barra al mantener `R` .
`src/main.js`	Es el **director de orquesta**. Une todos los archivos anteriores, controla el reloj, encierra el mapa en 2 pantallas cuando sale un jefe y se comunica con el backend para la tabla de posiciones.

B. Carpeta backend/ y Carpeta database/ (Servidor y Base de Datos)

Archivo	¿Qué hace explicado de forma simple?
`backend/config/cabecerasCors.php`	Configura los permisos del servidor para que el navegador pueda enviar y recibir datos en formato JSON sin bloqueos.
`backend/config/conexionBaseDatos.php`	Se conecta al motor MariaDB de XAMPP (usuario `root`) y crea automáticamente la base de datos `spiderman_db` y la tabla `puntajes` si todavía no existen.
`backend/config/funcionesAuxiliares.php`	Tiene dos funciones cortas para leer los datos que manda el formulario y devolver la respuesta en formato JSON.
`backend/api/puntajes/index.php`	Si recibe un pedido **GET**, devuelve el Top 20 de jugadores. Si recibe un pedido **POST**, valida el nombre, apellido, puntos (máx. 40.000) y tiempo, y los guarda en MariaDB.
`database/schema.sql`	Archivo SQL con las instrucciones en español para crear la base de datos `spiderman_db` y la tabla `puntajes` (con columnas `id` , `nombre` , `apellido` , `puntos` , `tiempo_segundos` , `fecha`).

4. ¿Cómo funcionan los Sprites en la página? (Explicación Súper Básica)

¿Qué es un Sprite? Un sprite es simplemente una imagen 2D con fondo transparente (PNG) de un personaje. Para que parezca que camina, salta o golpea, el juego funciona igual que un **dibujo animado o un folioscopio (flipbook)**: muestra varias fotos seguidas muy rápido (8 fotos por segundo).

1. **Organización en carpetas (`frontend/assets/sprites/`):** Cada enemigo y cada jefe tiene su propia carpeta ordenada con sus fotos separadas por acción: estar quieto (`Idle`), caminar (`Walk`), atacar (`Attack`) y recibir daño (`Damage`). En total hay **440 imágenes PNG**.
2. **El Mapa de Sprites (`assets_data.js`):** Como sería muy difícil escribir el nombre de 440 archivos a mano en el código, `assets_data.js` le asigna un "apodo" corto a cada foto (ej. `spr_boss_kingpin_walk_0`) y le indica a Phaser en qué carpeta está guardada.
3. **Cambio automático de foto según lo que hace el personaje:** En cada instante, `player.js` y `enemies.js` revisan qué hace el personaje: si está quieto muestra las fotos `idleKeys` , si camina pasa rápido las fotos `walkKeys` , si golpea muestra `atkKeys` , y si mira a la izquierda espeja la imagen con `setFlipX(true)` .
4. **El punto de apoyo en los pies (Para que nadie se hunda en la calle):** Todos los personajes tienen su anclaje en la planta de los pies: `setOrigin(0.5, 1.0)` . Así, aunque cambie el tamaño del dibujo en cada foto, **los pies siempre quedan apoyados sobre la calle (Y = 1400)**.